

Computer Vision

RDFIA Section2

Zhang Zhile & Moubarak Carine



Sorbonne Université
M2 DAC
December 24, 2024

Contents

1	Transfer Learning through feature extraction from a CNN	2
1.1	Section 1 – VGG16 Architecture	2
1.2	Section 2 – Transfer Learning with VGG16 on 15 Scene	4
1.2.1	Approach	4
1.2.2	Feature Extraction with VGG16	5
1.2.3	Training SVM classifiers	6
1.2.4	Going further	6
2	Visualizing Neural Networks	9
2.1	Section 1 – Saliency Map	9
2.2	Section 2 – Adversarial Examples	10
2.3	Section 3 – Class Visualization	11
3	Domain Adaptation	15
3.1	DANN and the GRL layer	15
3.2	Experiments	16
4	Generative Adversarial Networks	18
4.1	Generative Adversarial Networks	18
4.1.1	General Principle	18
4.1.2	Architecture of the networks	19
4.2	Conditional Generative Adversarial Networks	26
4.2.1	General Principle	26

Chapter 1

Transfer Learning through feature extraction from a CNN

1.1 Section 1 – VGG16 Architecture

- ***Q1:** Give an estimate on the number of parameters of VGG16.

To estimate the number of trainable parameters in VGG16, we focus on the fully connected layers, as they account for the majority of the parameters.

- **First Fully Connected Layer (FC6):** The input to this layer is of size $7 \times 7 \times 512 = 25,088$. With 4,096 neurons, the number of weights is:

$$(25,088 + 1) \times 4,096 = 102,764,544 \text{ parameters.}$$

- **Second Fully Connected Layer (FC7):** This layer receives an input of 4,096 from the previous layer and outputs 4,096 features. The number of weights is:

$$(4,096 + 1) \times 4,096 = 16,781,312 \text{ parameters.}$$

- **Third Fully Connected Layer (FC8):** This layer receives 4,096 inputs from the previous layer and outputs 1,000 features (corresponding to the 1,000 ImageNet classes). The number of weights is:

$$(4,096 + 1) \times 1,000 = 4,097,000 \text{ parameters.}$$

- **Total for Fully Connected Layers:** Summing up the parameters from all three fully connected layers, we have:

$$102,764,544 + 16,781,312 + 4,097,000 = 123,642,856 \text{ parameters.}$$

The fully connected layers alone account for approximately 89% of the total parameters in the model.

Convolutional Layers: The convolutional layers contribute significantly fewer parameters. Based on the architecture, the total number of parameters for the convolutional layers is:

14,714,688 parameters.

Total Parameters in VGG16: Adding the parameters from the fully connected and convolutional layers gives:

$123,642,856 + 14,714,688 = 138,357,544$ parameters.

The VGG16 model has approximately 138 million trainable parameters, with the fully connected layers accounting for the majority of these parameters.

- ***Q2:** What is the output size of the last layer of VGG16? What does it correspond to?

The output size of the last layer of VGG16 is 1000, which corresponds to the 1000 classes of the ImageNet dataset that the model was trained on. Each element in this output vector represents the network's prediction score for a specific class in the ImageNet dataset. After applying a softmax activation function, these scores are converted into probabilities. The class with the highest probability is considered the predicted class for the given input image.

- ***Q3:** Apply the network on several images of your choice and comment on the results.

add photos

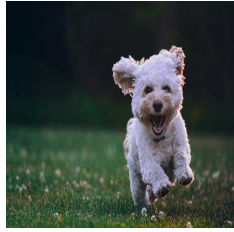
- What is the role of the ImageNet normalization?
- Why setting the model to eval mode?

We tested VGG16 with images of an Egyptian cat, a wire-haired fox terrier, a golden retriever, and a fox squirrel, and it correctly predicted each one. This shows how well the model can classify diverse animals from the 1,000 ImageNet classes.

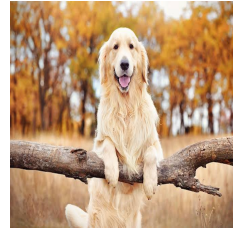
- ImageNet normalization plays a role in ensuring that the input images are preprocessed in a way that aligns with the conditions under which the model was trained. By adjusting the RGB values of the input image, it centers and standardizes the pixel values. This helps the model process the data more consistently and reduces numerical instabilities during computation. Without normalization, the input data distribution may differ from the training data.
- Setting the model to evaluation mode (`model.eval()`) is critical during inference as it ensures that layers like batch normalization and dropout behave correctly. In eval mode, batch normalization uses the pre-computed training statistics, and dropout is disabled, ensuring stable and consistent predictions.



(a) Egyptian cat



(b) wire-haired fox terrier



(c) golden retriever



(d) fox squirrel, eastern fox squirrel, *Sciurus niger*

Figure 1.1: Images used for testing VGG16

- ***Q4:** Visualize several activation maps obtained after the first convolutional layer. How can we interpret them?

The image shows the outputs of 64 different filters from the first layer of a neural network (VGG16). Each filter focuses on different parts of the input image, such as edges, textures, or specific areas. Bright highlights in the activations show where the filters detected important features, like the outline of the animal or details in the background. This helps the network identify key patterns in the image for further processing.

1.2 Section 2 – Transfer Learning with VGG16 on 15 Scene

1.2.1 Approach

- ***Q5:** Why not directly train VGG16 on 15 Scene?

The 15 Scene dataset is too small for training VGG16 from scratch, as the model's 138 million parameters require a large dataset like ImageNet to generalize well and avoid overfitting. Additionally, training VGG16 from scratch is computationally expensive and time-consuming, making it impractical for such a small dataset.

- ***Q6:** How can pre-training on ImageNet help classification for 15 Scene?

Pre-training on ImageNet helps with the classification of the 15 Scene dataset because the model has already learned a wide range of features, from detecting edges and textures to recognizing complex patterns. These features provide a strong starting point for understanding the 15 Scene images, even if the specific categories differ. This approach reduces the risk of overfitting on the smaller 15 Scene dataset and speeds up training by focusing on fine-tuning the pre-learned features instead of training the model from scratch.

- **Q7:** What limits can you see with feature extraction?

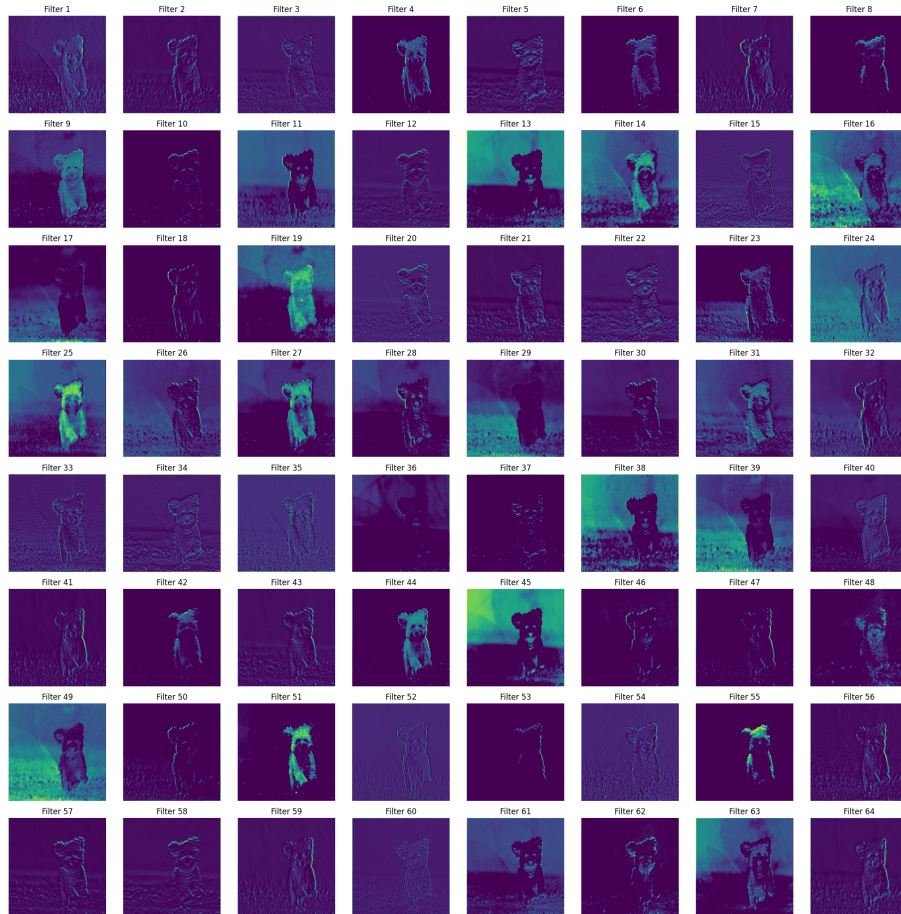


Figure 1.2: Activation Maps from the First Convolutional Layer of VGG16 on a Dog Image

The effectiveness of feature extraction really depends on how similar the pre-training task is to the target task. If the two tasks are very different (like using a model trained on natural images to analyze medical images or satellite...), the features might not be very useful. In that case, we might need to fine-tune the model or even retrain parts of it with data specific to the new task, which can be expensive and time-consuming. Also, if the pre-training dataset has biases, those can affect how well the features work for the new task. Lastly, models like VGG16 are pretty big and need a lot of computational power, which can be a problem if we are working on a device with limited resources.

1.2.2 Feature Extraction with VGG16

- **Q8:** What is the impact of the layer at which the features are extracted?

The layer at which features are extracted impacts the abstraction and specificity of the features. Early layers capture general features like edges and textures, making them adaptable across tasks, while deeper layers capture task-specific features. In VGG16, the first fully connected layer contains over 83% of the classifier's parameters, highlighting its role in shaping feature complexity. Choosing the right extraction layer balances generalization and specificity for the target task.

- **Q9:** The images from 15 Scene are black and white, but VGG16 requires RGB images. How can we get around this problem?

The images from the 15 Scene dataset are black and white, meaning they only have one channel, whereas VGG16 requires 3-channel RGB images. A simple solution is to replicate the single grayscale channel across all three RGB channels, creating a compatible input. Alternatively, the weights of the first convolutional layer in VGG16, which processes the RGB channels, can be averaged to adapt the model for grayscale inputs directly.

1.2.3 Training SVM classifiers

- **Q10:** Rather than training an independent classifier, is it possible to just use the neural network? Explain

Yes, it is possible to use the neural network itself instead of training a separate classifier. For example, with a model like VGG16, the final layer can be replaced with one that outputs 15 classes (for the 15 Scene dataset) and then fine-tuned. This approach works well for tasks that are similar to the original one. However, if the task is very different, the pre-trained classifier might not perform well, making fine-tuning or retraining necessary. Another option is to use a simpler classifier, like an SVM, with the features extracted from the network. This approach provides a good balance between computational efficiency and interpretability. The choice depends on the specific task and the available data.

1.2.4 Going further

- **Change the layer at which the features are extracted.**

Impact of Feature Extraction Layer Depth

The depth of the layer at which features are extracted significantly impacts the type, abstraction, and dimensionality of the features, which, in turn, affects classification performance and efficiency. Below is the analysis for three different layers of VGG16: `conv5_3`, `relu6`, and `relu7`.

Feature Extraction at `conv5_3`: The features extracted at this layer are very detailed and have a size of 25,088 (since there are 512 channels, each 7×7). These features keep a lot of spatial details, like edges and textures, which makes them good for tasks that need fine localization. But because of their high dimensionality, they require more computational power, which led to an SVM training time of 2.02 seconds. The accuracy achieved with these features was 0.8988.

Feature Extraction at `relu6`: The features extracted from `relu6` are compact and represent high-level patterns, with a size of 4,096. These features provide a good balance between detailed and abstract information, which makes them great for classification tasks. This layer had the best accuracy, reaching 0.9015, and the

SVM training time was shorter, taking only 1.60 seconds.

Feature Extraction at relu7: Features from `relu7`, with the same representation size of 4,096, are even more abstract and compact. While this abstraction reduces the risk of overfitting, some fine details may be lost, resulting in a slightly lower accuracy of 0.8858. The SVM training time was the fastest at 1.35 seconds, making this layer efficient for tasks requiring minimal computational cost.

Conclusion: The selection of the feature extraction layer depends on the specific task. For tasks like segmentation, where spatial details are critical, `conv5_3` is preferable. For classification tasks like 15 Scene, `relu6` offers the best accuracy-efficiency trade-off. `relu7` is suitable when computational efficiency is prioritized over slight accuracy losses.

- Try other available pre-trained networks

Comparison of Pre-trained Networks

We evaluated three pre-trained networks—VGG16, ResNet50, and DenseNet121—on feature extraction, training time, and accuracy. Below are the results and key observations.

Results Summary:

- **VGG16:** Feature extraction time: 38.45 seconds. Representation size: 4,096. SVM training time: 1.48 seconds. Accuracy: 88.58%.
- **ResNet50:** Feature extraction time: 23.36 seconds (faster than VGG16). Representation size: 2,048 (more compact). SVM training time: 2.90 seconds. Accuracy: 90.85%.
- **DenseNet121:** Feature extraction time: 25.13 seconds. Representation size: 50,176 (largest). SVM training time: 18.14 seconds. Accuracy: 91.96% (highest).

Feature Extraction Time: Modern architectures like ResNet50 and DenseNet121 are faster than VGG16, thanks to optimizations like residual connections in ResNet and feature reuse in DenseNet.

Representation Size:

- **VGG16:** Produces compact features of size 4,096, resulting from its fully connected layers.
- **ResNet50:** Outputs a compact 2,048-dimensional vector using global average pooling.
- **DenseNet121:** Produces very high-dimensional features (50,176) due to its dense connectivity, making it computationally expensive.

SVM Training Time: DenseNet121 has the longest training time due to its high-dimensional features, while ResNet50 and VGG16 are faster due to their compact representations.

Accuracy:

- **DenseNet121:** Achieved the highest accuracy (91.96%) due to its rich and dense feature representations.
- **ResNet50:** Achieved high accuracy (90.85%) with its residual connections aiding feature learning.
- **VGG16:** Slightly lower accuracy (88.58%), as it lacks modern architectural innovations like residual or dense connections.

Network Differences:

- **VGG16:** Sequential design with stacked convolutional and fully connected layers. Simple and effective but slower and less efficient compared to modern networks.
- **ResNet50:** Utilizes residual connections for efficient gradient flow, resulting in compact representations and fast feature extraction.
- **DenseNet121:** Employs dense connectivity for feature reuse, producing rich representations but at the cost of computational efficiency.

Conclusion:

- For speed and efficiency, use **ResNet50**, which balances compact features and accuracy.
- For maximum accuracy, use **DenseNet121**, especially if computational resources are available.
- For simplicity or legacy tasks, **VGG16** is a suitable option but less efficient than modern architectures.

- **Tune the parameter C to improve performance.**

The performance of the SVM classifier was evaluated with different values of the regularization parameter C . For $C = 1.0, 10$, and 100 , the accuracy remained constant at 88.58%, indicating that increasing C did not improve performance. The best accuracy of 88.91% was achieved with $C = 0.001$, suggesting that a simpler decision boundary generalizes better for this dataset. This highlights the importance of regularization in avoiding overfitting while maintaining robust generalization. For this task, $C = 0.001$ is the optimal choice.

Chapter 2

Visualizing Neural Networks

2.1 Section 1 – Saliency Map

- ***Q1:** Show and interpret the obtained results.

The results from Squeezenet1_1 show that the model focuses on very specific parts of the images when making predictions. For example, in the quail image, it highlights the bird's head, while in the dog or bear images, it focuses on the main body of the animals. This means Squeezenet relies on small, distinct details in the image rather than the whole scene. Its saliency maps are sparse, with only a few bright areas, showing that the model is efficient and uses only the most important features for classification.



Figure 2.1: Saliency Maps Highlighting Important Regions for Squeezenet Predictions

- **Q2:** Discuss the limits of this technique of visualizing the impact of different pixels.

Saliency maps show which parts of an image the model focuses on, but they don't explain why or if those parts truly influenced the decision. They can be noisy, unclear in complex images, and miss details due to limitations like ReLU saturation. They're also subjective, meaning people might interpret them differently. Better methods like SmoothGrad or Grad-CAM can fix some of these problems.

- **Q3:** Can this technique be used for a different purpose than interpreting the network?

Saliency maps can be used to identify and highlight important objects or regions in an image. This makes them useful for tasks like image segmentation, automated tagging, and as a starting point for object detection. They can also improve data augmentation by ensuring transformations like cropping or scaling preserve key areas in the image. However, this technique is mainly suited for image data and is less effective in non-visual domains or models that combine different types of data, like text and images.

- **Q4:** Test with a different network, for example VGG16, and comment.

The results show clear differences between Squeezenet1_1 and VGG16 in how they highlight important parts of the images. Squeezenet1_1 focuses on small, specific areas, like the quail's head or the bear's body, which means it relies on fewer details to make predictions. On the other hand, VGG16 highlights larger areas, covering more of the object and some background, showing it uses more information from the image. This happens because Squeezenet1_1 is smaller and faster but focuses less on details, while VGG16 is bigger and better at capturing complex features. Overall, VGG16 seems more reliable for harder tasks, while Squeezenet1_1 works well for simpler cases.



Figure 2.2: Saliency Maps Highlighting Important Regions for vgg16 Predictions

2.2 Section 2 – Adversarial Examples

- ***Q5:** Show and interpret the obtained results.

The results show how small, invisible changes to an image can trick the neural network into misclassifying it. The first image is the original (correctly labeled as "quail"), while the second image is slightly modified and is now classified as "stingray." The third panel shows the differences between the two images, which are so small that they're barely noticeable. The fourth panel magnifies these differences 10 times, revealing the subtle changes that caused the network to be fooled. This demonstrates how sensitive neural networks can be to tiny, targeted

changes in the input.

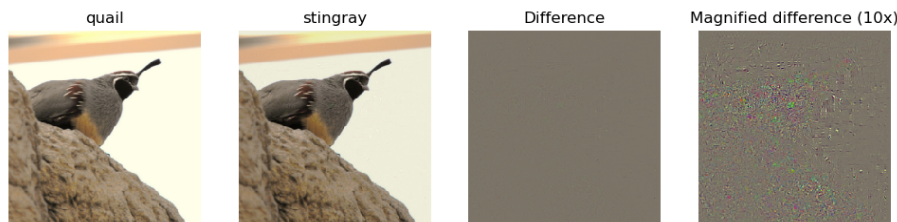


Figure 2.3

- **Q6:** In practice, what consequences can this method have when using convolutional neural networks?

This method shows how small changes can trick a model into making wrong predictions. While this might seem like a harmless experiment, it becomes dangerous in critical areas like self-driving cars or healthcare. For example, attackers could fool a car into misreading road signs or making unsafe decisions. Even without knowing the model's details, attackers can use "black-box" methods to confuse it. As machine learning becomes part of important systems, this creates serious risks, especially for safety and security. It highlights the need to protect models from these kinds of attacks.

- **Q7:** Discuss the limits of this naive way to construct adversarial images. Can you propose some alternative or modified ways? (You can base these on recent research).

One major limitation of the naive way to create adversarial images is that it often requires changing many pixels, making the modifications easier to detect and less practical. A better approach is to focus on changing only a small number of important pixels to fool the model while keeping the image changes subtle. Alternatives include using gradient-based methods like FGSM, which make minimal, targeted changes, or sparse attacks that identify and modify only the most critical pixels. Black-box attacks, which don't need access to the model's details, are also effective. These methods are more efficient, harder to detect, and better suited for real-world scenarios.

2.3 Section 3 – Class Visualization

- ***Q8:** Show and interpret the obtained results.

The results show how the model focuses on features it thinks represent a tarantula. Starting with random noise, the process changes the image step by step to make it match the tarantula class. By the final image, we see patterns like spider legs and textures that the model uses to recognize a tarantula, even though it doesn't look realistic to humans. This shows that the model doesn't see objects like we do, but focuses on certain patterns it has learned. Overall, the result shows how the model understands and identifies the tarantula class.

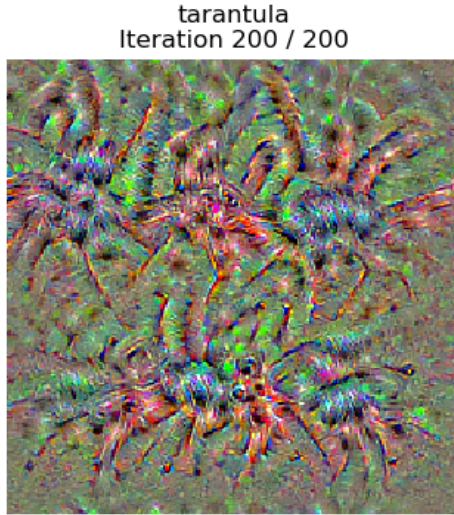
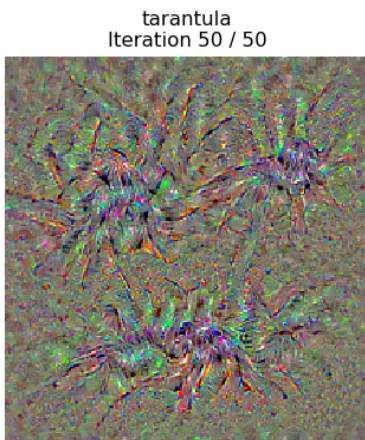


Figure 2.4: Class Visualization for Tarantula After 200 Iterations

- **Q9:** Vary the number of iterations, the learning and the regularization weight.

The images represent the model's visualization of the tarantula class after:

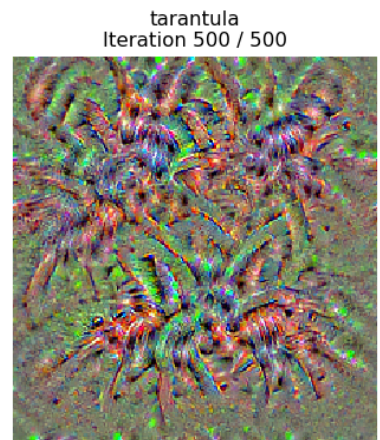
- **50 Iterations:** The features are rough and less defined. The tarantula patterns are beginning to emerge, but they are blurry and lack clear structure.
- **200 Iterations:** The patterns become more distinct and structured. The tarantulas are more recognizable, with clearer outlines of their legs and bodies.
- **500 Iterations:** The visualization becomes highly detailed, with well-defined tarantula features. The patterns are sharper, and the overall image is more consistent and focused. This indicates that the model has fully optimized for the tarantula class.



(a) 50 iterations

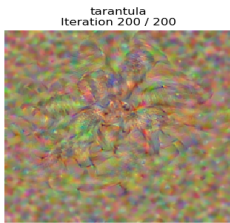


(b) 200 iterations

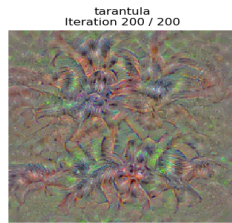


(c) 500 iterations

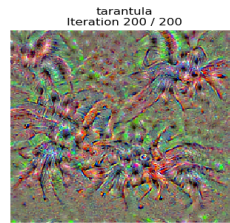
- **lr = 0.1:** The patterns are vague and less defined. The slower learning rate causes smaller updates, making the visualization evolve more gradually but failing to capture sharp features within 200 iterations.
- **lr = 1:** The tarantula features become clearer and more distinct. However, the moderate learning rate may not be fast enough to achieve the best clarity.
- **lr = 5:** The features are sharp, vibrant, and well-defined, with minimal noise. This learning rate has the best balance between steady optimization and clear visualization, making it the most effective choice.
- **lr = 10:** The tarantula patterns are detailed and well-defined, but there is also noticeable noise. A very high learning rate accelerates convergence but risks overshooting.



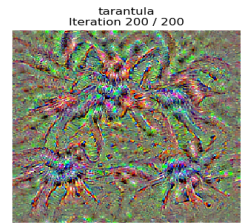
(a) lr = 0.1



(b) lr = 1

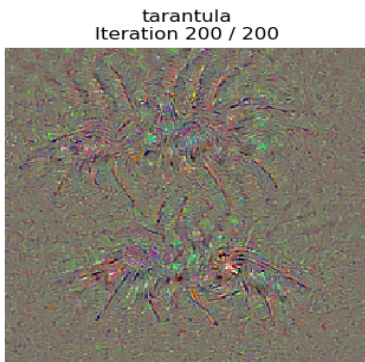


(c) lr = 5

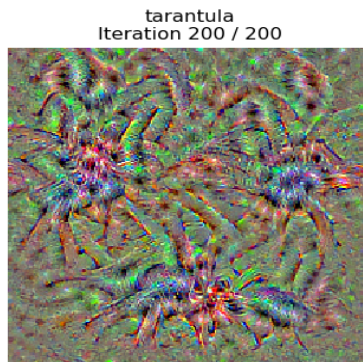


(d) lr = 10

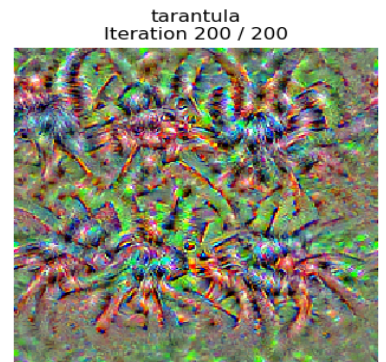
- **L2 = 0.01 :** The image looks smooth and blurry. The high regularization prevents too many changes, so the tarantula features are not clear.
- **L2 = 0.001 :** The tarantula features are more visible and detailed. This value keeps a good balance between detail and smoothness, making it the best choice.
- **L2 = 0.0001 :** The image has a lot of detail. Lower regularization allows bigger changes, which can make the image messy.



(a) l2 = 0.01



(b) l2 = 0.001

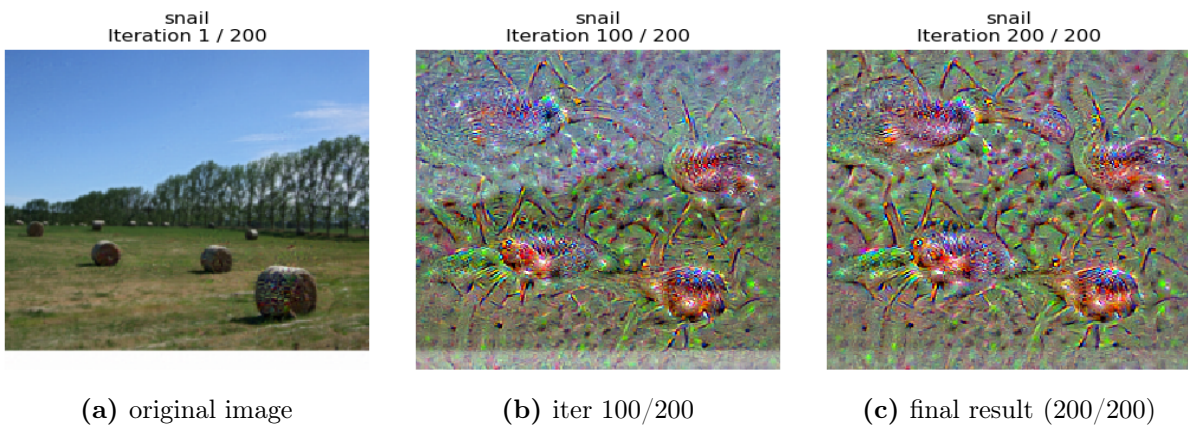


(c) l2 = 0.0001

- **Q10:** Try to use an image from ImageNet. Comment on the interest of doing this.

The refinement process for the "snail" class starting from an ImageNet image.

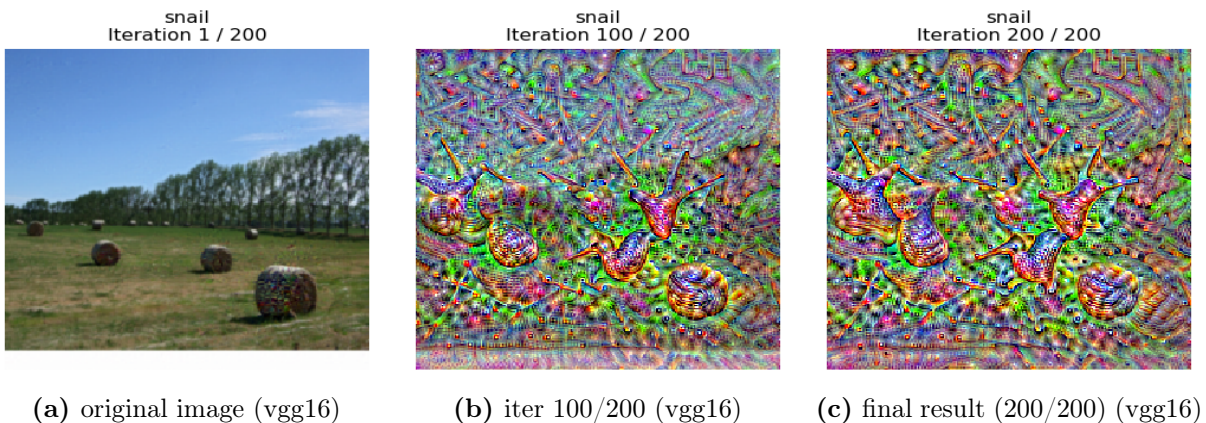
- Iteration 1: The visualization begins with the original ImageNet image (e.g., a natural scene). At this stage, no features specific to the snail class are visible.
- Iteration 100: Snail-like features start to emerge. Patterns resembling the snail's shape, body, and textures begin appearing. The hay bales further transform into elements that align with the snail class.
- Iteration 200: The visualization becomes sharper and more detailed. The snails are now clearly visible, with distinct shapes and textures, fully replacing the hay bales in the original image.



The interest in creating visualizations like this lies in understanding how neural networks interpret features for specific classes. It provides insights into the model's internal representations, showing patterns the model associates with a given class.

- **Q11:** Test with another network (VGG16), and comment on the results.

VGG16 produces more detailed and vibrant visualizations for the "snail" class due to its deeper architecture, but it also introduces more noise. While the features are sharper, the background becomes less clean and harder to interpret.



Chapter 3

Domain Adaptation

3.1 DANN and the GRL layer

- **Q1:** If you keep the network with the three parts (green, blue, pink) but didn't use the GRL, what would happen ?

Without the Gradient Reversal Layer (GRL), the domain classifier's gradient pushes the feature extractor to create features that make it easier to tell the source and target domains apart. This causes the model to become domain-specific, meaning it works well only for the source domain but poorly for the target domain. The GRL fixes this by flipping the gradient's direction, forcing the feature extractor to create domain-agnostic features that confuse the domain classifier. This helps the model focus on shared features between the source and target, achieving better domain adaptation.

- **Q2:** Why does the performance on the source dataset may degrade a bit ?

The performance on the source dataset may degrade slightly because the Gradient Reversal Layer (GRL) forces the model to focus on features that work for both the source and target domains, instead of just optimizing for the source domain. This trade-off helps the model generalize better to the target domain but slightly reduces its specialization for the source.

- **Q3:** Discuss the influence of the value of the negative number used to reverse the gradient in the GRL.

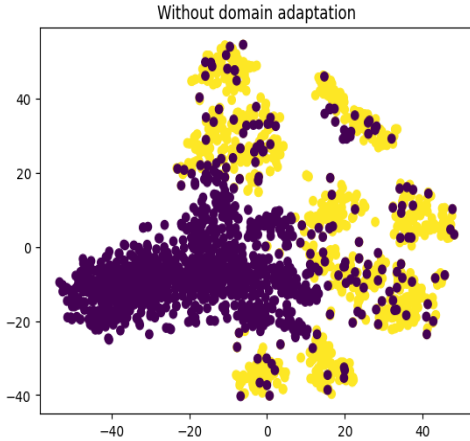
The value of the negative number in the Gradient Reversal Layer (GRL) determines the balance between domain adaptation and source domain performance. A small negative value, like -0.1 , applies a weak reversal, prioritizing source-specific features but resulting in poor target domain generalization. A large negative value, such as -10 , strongly penalizes domain-specific features, improving domain alignment but potentially degrading source performance by discarding important source-specific features. A moderate value, like -1 , provides a balance, encouraging domain-invariant features while retaining task-relevant ones. This

usually leads to slight source performance degradation but significantly better generalization to the target domain.

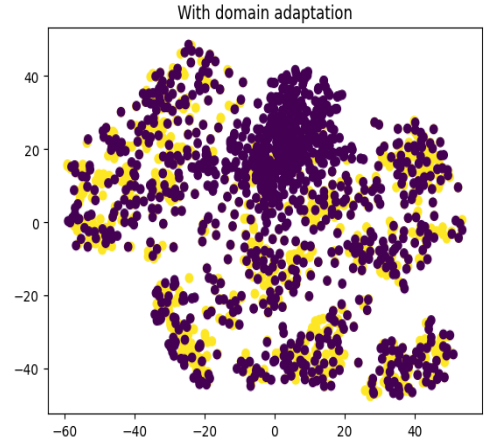
- **Q4:** Another common method in domain adaptation is pseudo-labeling. Investigate what it is and describe it in your own words.

Pseudo-labeling is a method in domain adaptation where the model guesses labels for the target domain data (which doesn't have real labels) and uses these guessed labels, called pseudo-labels, to improve its performance. First, the model is trained on the labeled source domain data. Then, it predicts labels for the target data and picks the predictions it is most confident about. These predictions are treated as if they are correct labels. Finally, the model is trained again using both the source data with real labels and the target data with pseudo-labels. This helps the model learn patterns in the target domain. But if the guessed labels are wrong, it can make the model worse. To avoid this, the model only uses predictions it is very confident about.

3.2 Experiments



(a) TSNE without domain adaptation



(b) TSNE with domain adaptation

1. Sigmoid-Based GRL Factor

We initially used the sigmoid-based GRL factor defined as:

$$\lambda(e) = -1 + \frac{2}{1 + \exp\left(-\frac{2e}{\text{nb_iters}}\right)}$$

– 20 Epochs:

- * **Source domain:** High classification accuracy (98.78%) and moderate domain classifier accuracy (77.63%).
- * **Target domain:** Classification accuracy was 79.22%, showing reasonable domain adaptation, but domain classifier accuracy was low (53.56%), indicating limited domain alignment.

- * **Observation:** The target domain accuracy was acceptable, but domain alignment could be improved.

– **50 Epochs:**

- * **Source domain:** Slight drop in classification accuracy (98.25%) with improved domain classifier accuracy (85.81%).
- * **Target domain:** Noticeable drop in classification accuracy to 74.2%, while domain classifier accuracy improved slightly to 57.05%.
- * **Observation:** Longer training introduced overfitting to the source domain, leading to reduced target performance despite improved domain alignment.

2. Cosine-Based GRL Factor

We replaced the sigmoid GRL factor with a cosine-based GRL factor defined as:

$$\lambda(e) = 0.5 \cdot \left(1 - \cos \left(\pi \cdot \frac{e}{\text{nb_iters}} \right) \right)$$

– **20 Epochs:**

- * **Source domain:** Classification accuracy remained high (98.76%) with significantly improved domain classifier accuracy (90.23%).
- * **Target domain:** Classification accuracy dropped slightly to 75.33%, and domain classifier accuracy was 50.27%.
- * **Observation:** The cosine GRL factor provided stronger domain alignment, as evidenced by higher domain classifier accuracy on the source, but the target classification accuracy did not improve as expected.

Chapter 4

Generative Adversarial Networks

4.1 Generative Adversarial Networks

4.1.1 General Principle

- **Question 1:** Interpret the equations (6) and (7). What would happen if we only used one of the two?

Equations (6) and (7) describe the optimization objectives for the generator and discriminator in a GAN.

$$\max_G \mathbb{E}_{z \sim P(z)} [\log D(G(z))]$$

This is the generator's objective. Its goal is to maximize the discriminator's confidence that generated samples $G(z)$ are real, encouraging the generator to produce more realistic data.

$$\max_D \mathbb{E}_{x^* \in \text{Data}} [\log D(x^*)] + \mathbb{E}_{z \sim P(z)} [\log(1 - D(G(z)))]$$

This is the discriminator's objective. It seeks to:

- Maximize $D(x^*)$, the confidence that real samples are classified as real.
- Minimize $D(G(z))$, reducing the confidence that generated samples are classified as real.

If only the generator's objective is used, the discriminator would not adapt to distinguish real and generated data, leaving the generator with inadequate feedback. Conversely, if only the discriminator's objective is used, the generator would not improve, and the discriminator would trivially classify all generated data as fake.

- **Question 2:** Ideally, what should the generator G transform the distribution $P(z)$ to?

The generator G should ideally take the prior distribution of the noise vector z denoted as $P(z)$, and transform it into the data distribution $P(X)$, which characterizes the real data samples.

The generator G learns a mapping from the noise space (latent space) to the data space such that the samples $G(z)$ it generates are indistinguishable from real samples according to the discriminator. As a result, the generator transforms the noise distribution $P(z)$ into an approximation of the real data distribution $P(X)$.

- **Question 3:** Remark that the equation (6) is not directly derived from equation (5). This is justified by the authors to obtain more stable training and avoid the saturation of gradients. What should the “true” equation be here?

From equation (5), the "true" objective for the generator G can be derived as:

$$\min_G \mathbb{E}_{z \sim P(z)} [\log(1 - D(G(z)))]$$

This equation indicates that the generator G aims to produce samples $G(z)$ such that the discriminator D outputs a value close to 1 for them. In other words, the generator tries to minimize the discriminator’s confidence that the generated samples are fake.

The authors adjusted the generator’s objective to the form in equation (6):

$$\max_G \mathbb{E}_{z \sim P(z)} [\log D(G(z))]$$

This adjustment was made to prevent vanishing gradients and improve training stability.

4.1.2 Architecture of the networks

- **Question 4:** Comment on the training of the GAN with the default settings. (progress of the generations, the loss, stability, image diversity, etc.)

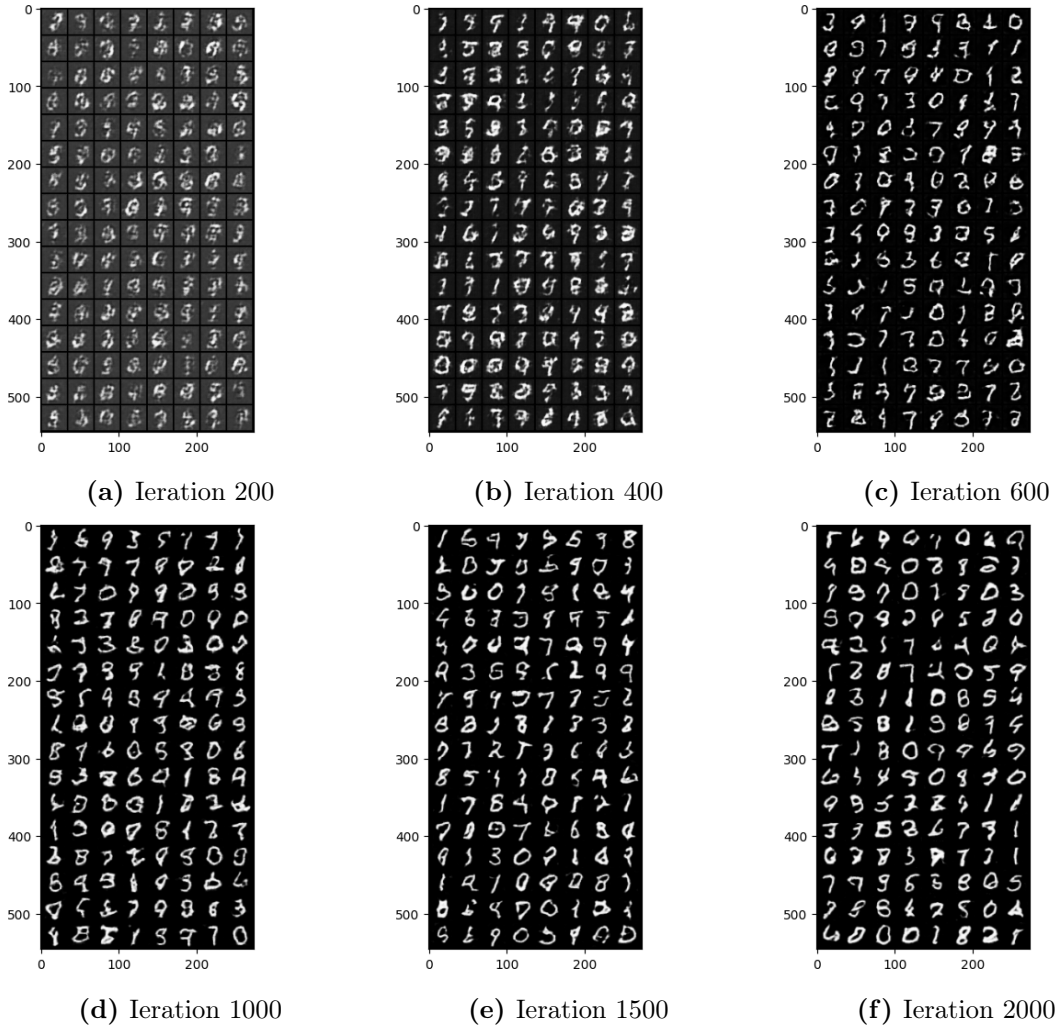
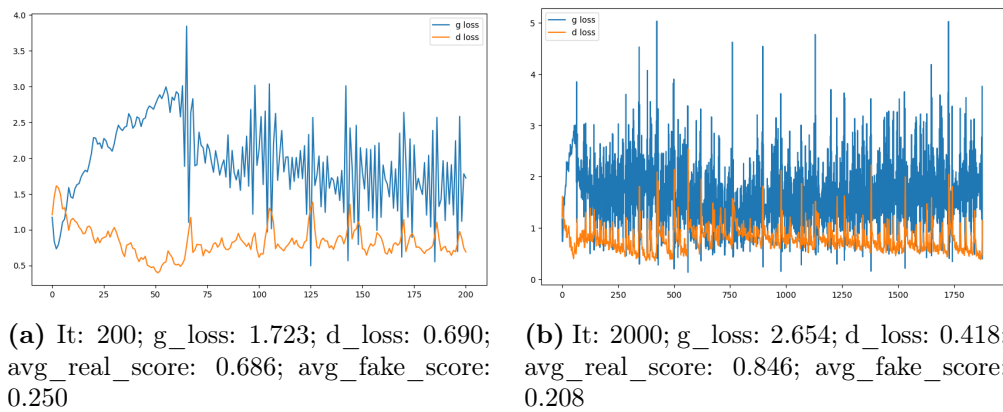


Figure 4.1: Generated Images By GAN From 200 Iteration To 2000 Iteration



1. Progress of Generated Image Quality

From the provided generated images, we can observe the following:

- From iteration 200 to 2000, the outputs of the generator gradually transitioned from blurry and unclear digit shapes to clearer and more realistic

distributions. This indicates that the generator is progressively learning the data distribution and generating more realistic samples.

- However, at iteration 2000, some generated digit images still show certain degrees of distortion or noise, which suggests that the generator has not fully mastered the target distribution and requires further optimization.

2. Analysis of Loss Curves

The loss curves for the generator (`g_loss`) and the discriminator (`d_loss`) reveal some abnormalities during the iterations:

- **Early stage (around iteration 200):**

The generator's loss is relatively low (1.723), and the discriminator's loss (0.690) is moderate. This might indicate that the discriminator has not yet fully learned to distinguish between real and generated samples.

`avg_real_score` is low (0.686), while `avg_fake_score` is relatively high (0.250), suggesting that the discriminator is not accurate enough in identifying real samples and is overly lenient with generated samples. This shows that both the discriminator and generator are undertrained at this stage.

- **Later stage (around iteration 2000):**

The generator's loss increases significantly (2.654), while the discriminator's loss decreases (0.418). This is a common phenomenon, indicating that the generator is striving to improve its samples to deceive the discriminator, but the discriminator simultaneously becomes stronger, leading to a higher generator loss.

`avg_real_score` rises to 0.846, reflecting an improvement in the discriminator's accuracy in identifying real samples. However, `avg_fake_score` drops to 0.208, showing that the discriminator has become stricter with generated samples, and the generator's improvements are not sufficient to fully deceive the discriminator.

3. Instability in GAN Training Dynamics

The losses of the generator and discriminator fluctuate significantly throughout the training process, which might be caused by an imbalance in training dynamics. The discriminator learns faster than the generator, making it difficult for the generator to catch up in the early stages, resulting in an increased generator loss in the later stages.

4. Diversity of Generated Images

Although the generated images become clearer, the structural characteristics of the samples show some similarities (e.g., certain digit shapes remain fixed). This might indicate that the generator may have only learned certain high-frequency patterns from the training data while failing to cover the entire data distribution comprehensively.

- **Question 5:** Comment on the diverse experiences that you have performed with

the suggestions above. In particular, comment on the stability of training, the losses, and the diversity of generated images.

To improve the training of the Conditional GAN, we experimented with the following modifications:

- Increasing the number of training epochs.
- Increasing the number of generator features (`ngf`).
- Changing the learning rate.
- Testing the model on a new dataset CIFAR-10.

1. Modifying `ngf` and Increasing the Number of Epochs to 30

The parameter `ngf` controls the number of feature maps in the generator. By increasing `ngf`, the generator can learn more complex patterns and improve the quality of the generated samples. I experimented with increasing `ngf` from 32 to 128, and observed significant improvements in the clarity and diversity of the outputs.

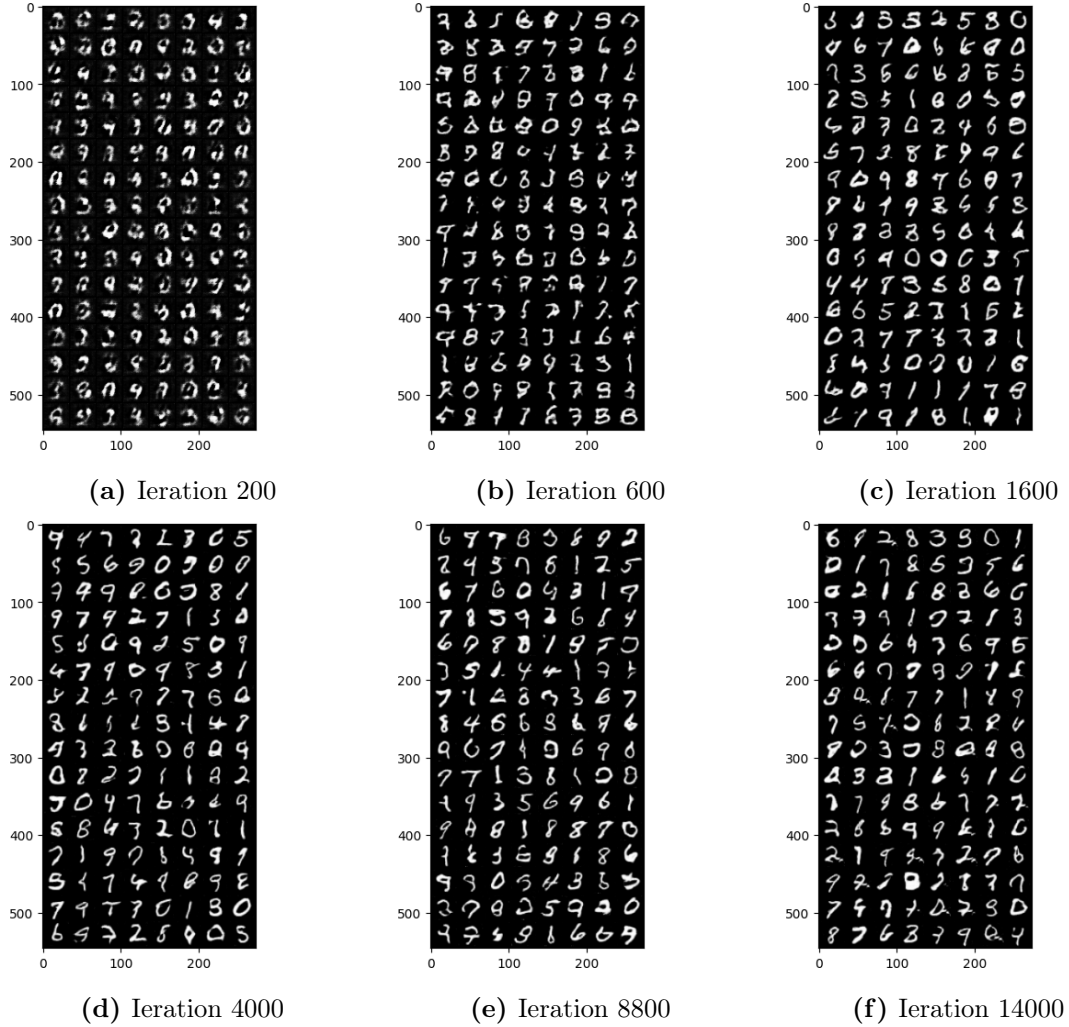
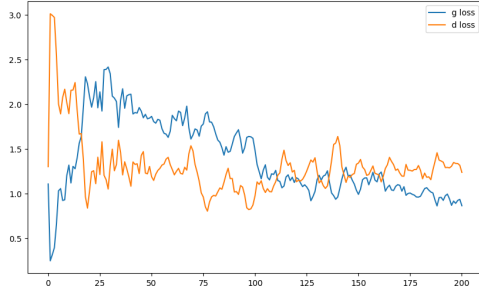
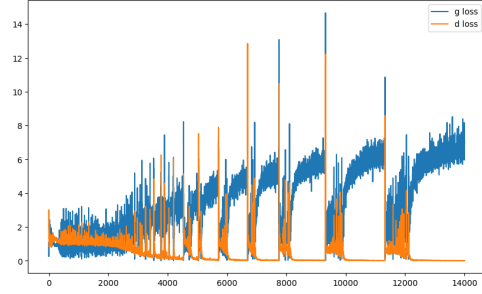


Figure 4.3: Modifying `ngf`: Generated Images By GAN From 200 To 14,000 Iteration



(a) It: 200; g_loss: 0.866; d_loss: 1.238;
avg_real_score: 0.530; avg_fake_score: 0.435



(b) It: 14069; g_loss: 6.760;
d_loss: 0.003; avg_real_score: 0.998;
avg_fake_score: 0.002

In this experiment, we chose to increase (**ngf**) to enhance the generator's feature extraction capability, allowing it to generate higher-quality samples. Because of the previous training results with default parameters, we found the **avg_fake_score** approached 0 in the later stages and the generator loss (**g_loss**) continued to increase. This indicates that the discriminator became overly strong in the later stages, making it difficult for the generator to produce realistic enough samples to compete with it. Therefore, it was necessary to enhance the learning ability of the generator.

The experimental results show that the generator loss (**g_loss**) exhibits a pattern of "stage-wise jumps, sudden drops, and subsequent rises." This is a common dynamic phenomenon in GAN training: as the discriminator becomes more powerful (discriminator loss approaches 0), the generator loss increases significantly, reflecting the generator's difficulty in producing realistic samples to fool the discriminator. Subsequently, the generator improves the quality of its samples and temporarily succeeds in deceiving the discriminator, causing the loss to drop. However, as the discriminator regains its advantage, the generator loss rises again. This loss fluctuation indicates a dynamic balance between the generator and the discriminator, where each alternately gains the upper hand. Meanwhile, the discriminator loss (**d_loss**) decreases to near zero, showing that the discriminator becomes extremely accurate in distinguishing real samples from generated ones.

From the perspective of generated samples, as the number of iterations increases, the clarity and detail of the images improve significantly. Particularly in the later stages, the generated digits become clearer and structurally complete, with a substantial reduction in blurriness and distortion. Complex digits, such as "8" and "9," exhibit especially notable improvements in their structures.

However, in the later iterations, the discriminator's excessive strength still limits the generator's room for improvement. This suggests that although the generator has improved, it still does not fully approximate the true data distribution.

2. Changing the Learning Rate:

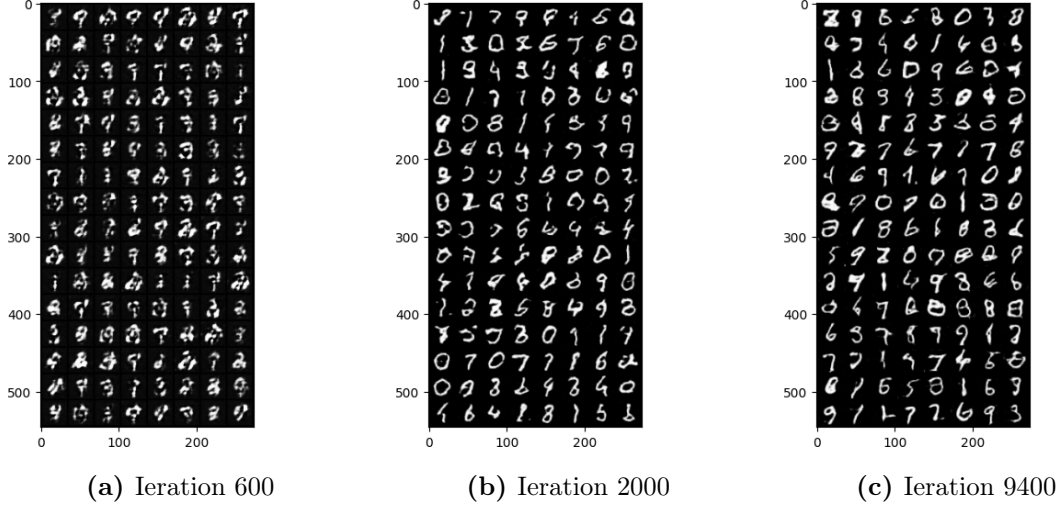
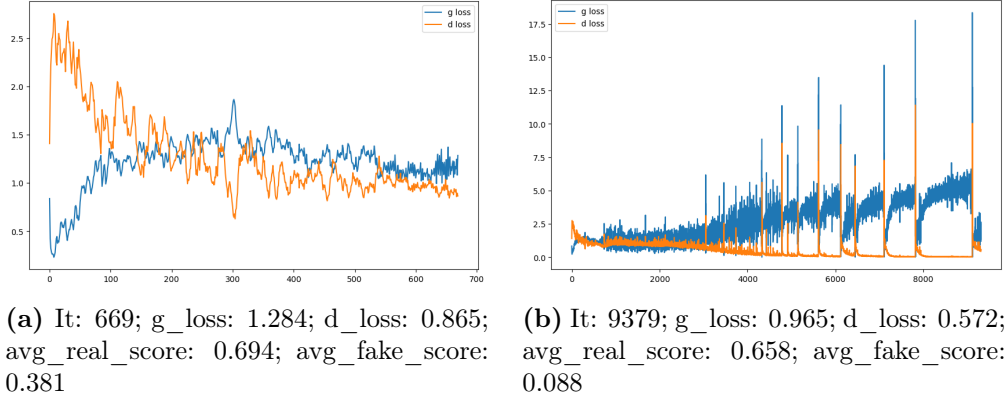


Figure 4.5: Changing Lr: Generated Images By GAN From 200 To 9400 Iteration



After modifying the learning rates of the generator and discriminator, the experimental results show a trend similar to the experiment with adjusted `ngf`, but the speed of generating high-quality images is noticeably slower. For example, at iteration 600, the generated images are still blurry, and it is only after iteration 2000 that they gradually become clearer. This probably because the generator's learning rate was increased to 0.0007 from 0.0005, accelerating its update steps but also causing instability in the generator's outputs in the early stages. Meanwhile, the discriminator's learning rate was reduced to 0.0001 from 0.0002, this slows down its optimization speed which led to insufficient adversarial dynamics in the initial stages.

According to the loss curves, the adversarial relationship between the generator and discriminator appears more stable, with smaller jumps. However, the slower improvement of the generator suggests that this may not be an optimal modification, we haven't found a better combination of learning rates yet.

3. Testing on CIFAR-10 Dataset:

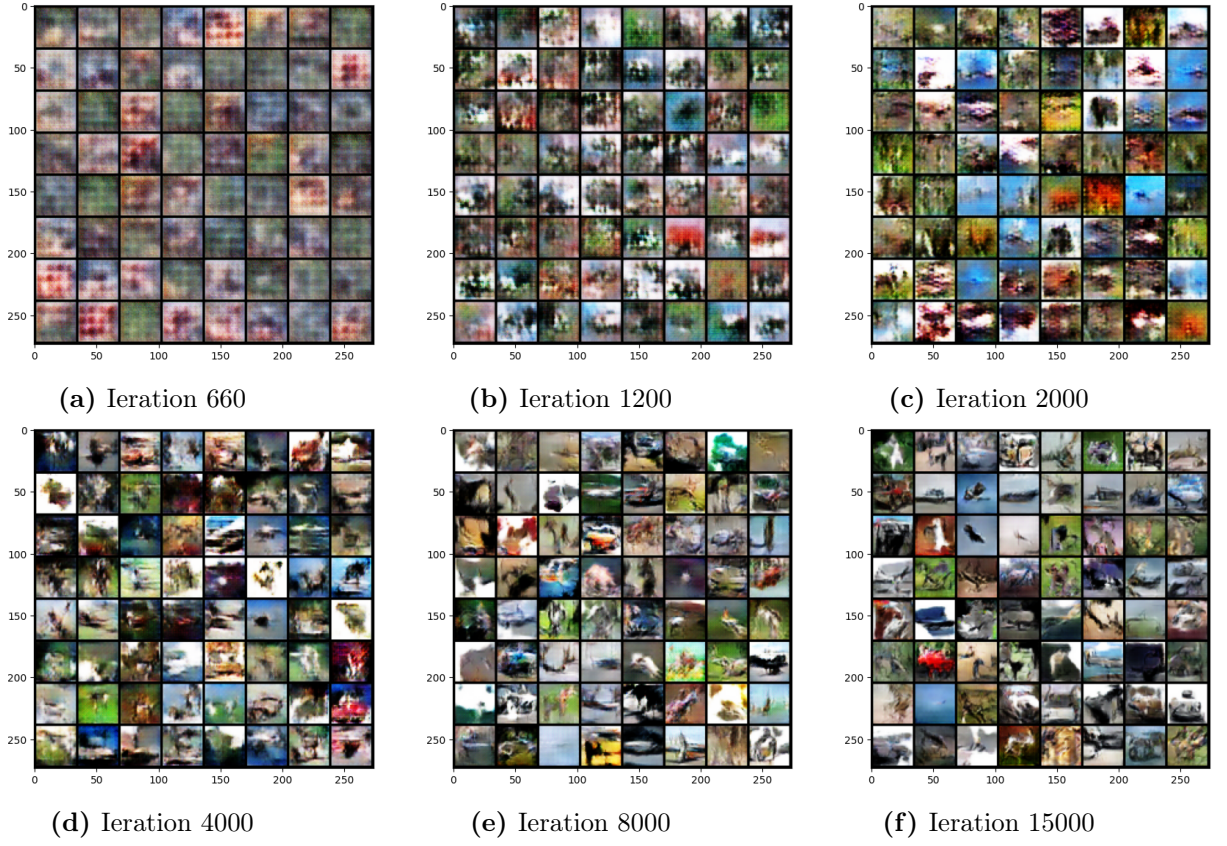
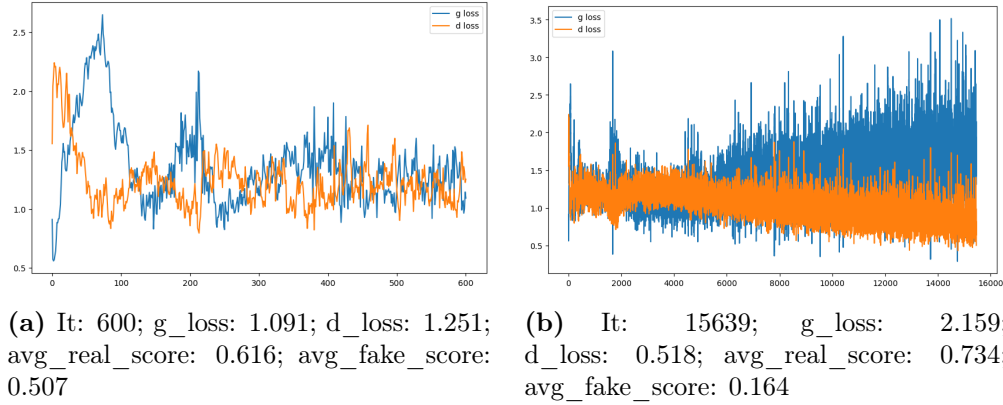


Figure 4.7: CIFAR-10: Generated Images By GAN From 200 To 15,000 Iteration



The experimental results on the CIFAR-10 dataset show that the improvement in the quality of generated images is significantly slower compared to the MNIST dataset. At early iterations, the generated images on CIFAR-10 remain blurry and difficult to discern while MNIST at the same number of iterations has already produced relatively clear digits. By iteration 15,000, the generated images on CIFAR-10 gradually exhibit the basic shapes of objects, but with some samples still showing noise and blurriness, the quality and details are still inferior to those on MNIST. This is due to the greater complexity of the categories and features in CIFAR-10, which significantly increases the learning difficulty for the generator.

From the loss curves, the g_loss and d_loss on CIFAR-10 fluctuate less, and

the training is more stable, without significant jumps or drops. This is because the complexity of CIFAR-10's data distribution limits the generator's short-term improvement, while the adversarial dynamics between the generator and discriminator are more balanced. However, the generator's improvement is relatively slow. The *avg_fake_score* on CIFAR-10 decreases from 0.507 to 0.164, lower than the closer-to-zero scores in the MNIST experiment, indicating that the generator finds it harder to produce high-quality samples to deceive the discriminator. Meanwhile, the *avg_real_score* increases from 0.616 to 0.734, showing that the discriminator's ability to judge real samples is also weaker than on MNIST. In comparison, the complexity of CIFAR-10 data makes the adversarial training more challenging.

4.2 Conditional Generative Adversarial Networks

4.2.1 General Principle

- **Question 6:** Comment on your experiences with the conditional DCGAN.

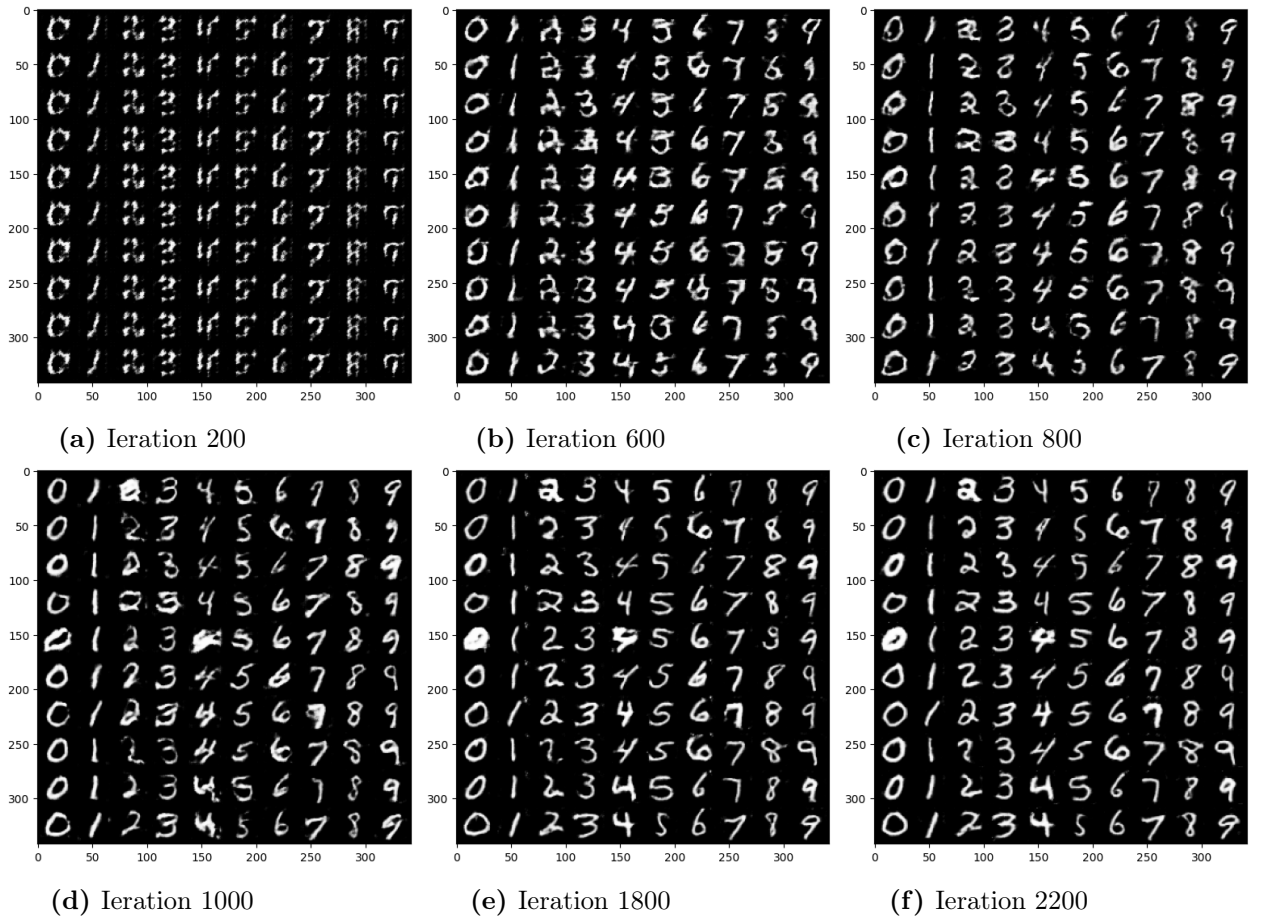
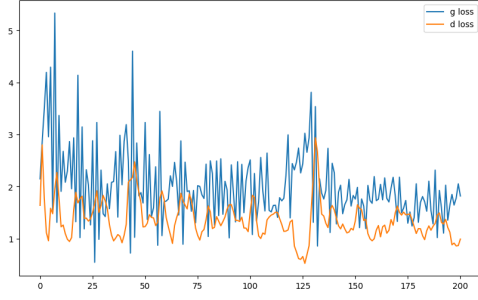
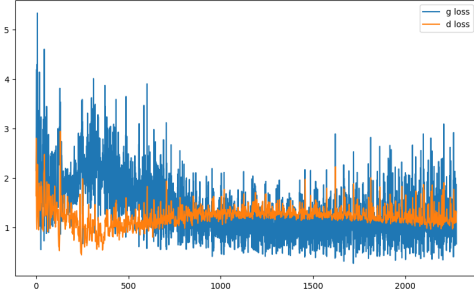


Figure 4.9: Generated Images By Conditional GAN From 200 To 2000 Iteration



(a) It: 200; g_loss: 1.820; d_loss: 0.991;
avg_real_score: 0.634; avg_fake_score:
0.349



(b) it: 2000; g_loss: 2.099; d_loss: 1.171;
avg_real_score: 0.804; avg_fake_score:
0.593

1. Progress of Generated Image Quality

- **Early stage:** The generated images are overall blurry, with unclear contours of digits, and some digits are unrecognizable. For example, digits "3" and "8" appear distorted and difficult to distinguish.

Some digits lack complete features, indicating that the generator has not yet learned how to produce high-quality samples. The effect of the condition y is insufficient at this stage. In some rows, the digit order is disorganized or repetitive. For example, multiple "7"s appear in the same row which means the generator has not effectively utilized the conditional information.

- **Mid-stage:** The digit contours become clearer, and some digits (such as "1" and "0") begin to look more realistic, but complex digits (such as "5" or "8") still show some distortion or missing parts.

The image brightness improves, and the generator begins to generate samples with better pixel intensity. The effect of the condition y becomes more apparent, and digits in different conditions match the expected order more closely. For example, the digits in each row mostly follow the conditional label sequence. However, minor errors still occur, such as a "2" appearing where a "3" should be.

- **Later stage:** The clarity and detail of the digits significantly improve, with complete and well-defined contours. For example:

- The vertical and horizontal lines of "4" are well-formed and proportional.
- The loops in "8" are clear, with minimal noise interference.

The image quality approaches that of real samples, making it much harder for the discriminator to distinguish between real and generated samples. The conditional y becomes fully effective, and digits generated in each row strictly follow the expected order without obvious errors.

There is good diversity under the same condition. For instance, digits labeled "7" exhibit slight variations in font thickness or tilt, demonstrating the generator's ability to produce diverse outputs.

2. Analysis of Loss Curves

- **Generator Loss (g_loss):** In the early stage, the generator loss is relatively low (1.820), indicating that the generator has initially managed to produce some samples that the discriminator partially accepts.

By the later stage, the generator loss increases to 2.099. This can be caused by the discriminator becoming stronger, requiring the generator to put in more effort to produce realistic samples and providing more precise gradient signals, pushing the generator to generate higher-quality outputs.

- **Discriminator Loss (d_loss):** In the early stage, the discriminator loss is moderate (0.991), indicating limited ability to distinguish real from generated samples.

By the later stage, the discriminator loss increases to 1.171, suggesting that the generator's outputs are approaching the real data distribution, increasing the difficulty for the discriminator to differentiate.

- **Average Scores :**

$avg_real_score = 0.634$ to $avg_real_score = 0.804$ means in the beginning, the discriminator's judgment of real samples is not very accurate, indicating undertraining at this stage, at the end it's accuracy in identifying real samples improves significantly.

$avg_fake_score = 0.349$ to $avg_fake_score = 0.593$ means the discriminator in the beginning can easily identify low-quality generated samples and then the quality of generated samples increases, making it harder for the discriminator to differentiate and suggesting that the generator is approaching the real sample distribution.

- **Question 7:** Could we remove the vector y from the input of the discriminator (so having $cD(x)$ instead of $cD(x, y)$)?

No, if the conditional vector y is removed from the discriminator's input, the conditional generation capability of the Conditional GAN will be completely lost. The discriminator will no longer be able to verify whether the generated samples satisfy the condition y , thereby failing to provide condition-based feedback to the generator. As a result, the generator's output will no longer be constrained by the condition y , potentially generating random or irrelevant samples.

Moreover, the adversarial training between the generator and discriminator would lose a crucial conditional feedback loop, potentially destabilizing the training process or preventing convergence altogether.

- **Question 8:** Was your training more or less successful than the unconditional case? Why?

The results show that Conditional GAN's training more successful than the unconditional GAN in terms of sample quality, consistency, and training dynamics. By

utilizing the conditional vector y , cGAN produces samples that gradually transition from blurry to clear and strictly match the specified conditions. For instance in the generated images, each row corresponds accurately to the digits from 0 to 9 based on the conditional labels. As the number of iterations increases, the generated samples become progressively more detailed and ultimately achieving high clarity. In contrast, the unconditional GAN lacks conditional constraints, which results more randomness in the outputs. This lack of guidance leads to poorer quality and inconsistency compared to cGAN.

From the perspective of scores and loss curves, cGAN demonstrates more stable training dynamics. In the early stage, the generator loss (g_loss) is 1.820 and the discriminator loss (d_loss) is 0.991, indicating that the generator can already produce some samples that are partially accepted by the discriminator, but the discriminator still has room for improvement. At this point, avg_real_score is 0.634 and avg_fake_score is 0.349, showing that the discriminator's judgment of real samples is not yet highly accurate, while distinguishing generated samples is relatively easy. By the later stage, the generator loss increases to 2.099 and the discriminator loss rises to 1.171, this reflecting that the discriminator has become stronger and requiring the generator to produce higher-quality samples to fool it. At the same time, avg_real_score improves to 0.804 and avg_fake_score increases to 0.593, indicating that the quality of the generated samples has significantly improved, making it more difficult for the discriminator to distinguish between real and fake samples.

In contrast, the loss curves and scores for the unconditional GAN typically exhibit greater fluctuations and instability. Without the feedback provided by the conditional vector y , the generator in the unconditional GAN shows slower progress in improving sample quality and consistency, and the training process is more prone to imbalance.

- **Question 9:** Test the code at the end. Each column corresponds to a unique noise vector z . What could z be interpreted as here?

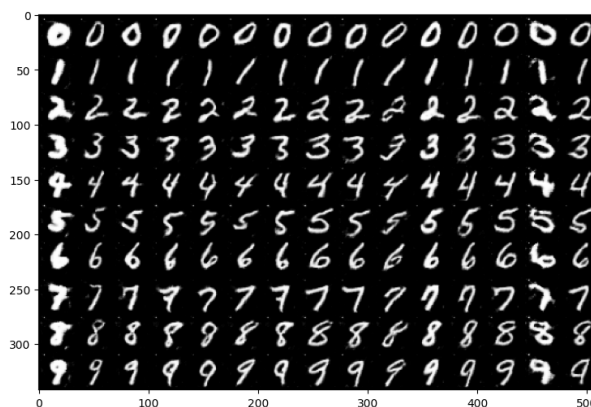


Figure 4.11

In this case, the noise vector z can be understood as a factor controlling the style of the generated samples. Within each column, z is fixed, so the digits in the same column share similar styles, such as stroke thickness, slant, or shape characteristics (roundness or sharpness). In other words, z determines the overall style or fine details of the generated images.

At the same time, z does not decide the digit category (this is the role of the conditional vector y). Each row corresponds to a specific condition y , which determines the digit class, such as "0" or "5". By varying z , the same digit category (e.g., "3") can be generated with diverse styles, such as different stroke thicknesses or angles. z is a latent vector used by the generator to introduce diversity in the generated samples, primarily controlling the style characteristics without affecting the digit class.